

GitHub Actions OIDC → Amazon ECR (push images)

Short setup for the FGS **dev** stack. CI pushes Setup, User, and nginx images into **one** ECR repository (fgs) using **OIDC**. Do **not** store AWS_ACCESS_KEY_ID or AWS_SECRET_ACCESS_KEY in GitHub.

Full deploy runbook: [MANUAL DEPLOY NGINX SETUP USER.md](#) (sections A6.3 and B1).

What you get

```
git push (dev / test / main)
→ GitHub Actions (Build setup | Build user | Build nginx)
→ OIDC assume role AWS_ROLE_TO_ASSUME
→ docker login to ACCOUNT.dkr.ecr.REGION.amazonaws.com
→ docker push ../fgs:setup-dev (also user-dev, nginx-dev)
```

Piece	Value
Region	Same as ECR (example: us-east-1)
ECR repository	fgs (one shared repo)
Image tags on dev	setup-dev, user-dev, nginx-dev
IAM role	fgs-dev-github-actions
GitHub variables	AWS_REGION, AWS_ROLE_TO_ASSUME, optional ECR_REPO

Prerequisite: create the private ECR repository **fgs** first (Amazon ECR → Private registry → Create repository).

Step 1 — OIDC identity provider (once per AWS account)

1. Open **IAM** → **Identity providers** → **Add provider**.
2. Provider type: **OpenID Connect**.
3. Provider URL: `https://token.actions.githubusercontent.com`
4. Click **Get thumbprint**.
5. Audience: `sts.amazonaws.com`
6. **Add provider**.

Skip if `token.actions.githubusercontent.com` already exists in the account.

Step 2 — Create the IAM role

1. **IAM** → **Roles** → **Create role**.
 2. Trusted entity type: **Web identity**.
 3. Identity provider: `token.actions.githubusercontent.com`.
 4. Audience: `sts.amazonaws.com`.
 5. Role name: `fgs-dev-github-actions`.
 6. Create the role (you can skip AWS managed policies for now).
-

Step 3 — Trust policy

Roles → `fgs-dev-github-actions` → **Trust relationships** → **Edit trust policy**.

Replace ACCOUNT with your AWS account ID. Set sub to your real GitHub org and repo:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::ACCOUNT:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:nathshivfsm-sys/fgs-coreapi:*"
        }
      }
    }
  ]
}
```

Wrong sub (wrong org or repo name) is the usual cause of:

Not authorized to perform `sts:AssumeRoleWithWebIdentity`

Step 4 — Permissions (push to ECR fgs)

Permissions → **Add permissions** → **Create inline policy** → **JSON**.

Replace ACCOUNT and REGION (example region `us-east-1`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:PutImage",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:DescribeRepositories",
        "ecr:DescribeImages"
      ],
      "Resource": [
        "arn:aws:ecr:REGION:ACCOUNT:repository/fgs"
      ]
    }
  ]
}
```

Name the policy e.g. ecr-push-fgs. Save.

The ECR repository `fgs` must already exist in that region.

Step 5 — Copy the role ARN

Example:

```
arn:aws:iam::286093098927:role/fgs-dev-github-actions
```

You will paste this into GitHub as `AWS_ROLE_TO_ASSUME`.

Step 6 — GitHub Actions variables

In the GitHub repo: **Settings** → **Secrets and variables** → **Actions** → **Variables** → **New repository variable**.

Variable	Value
AWS_REGION	Same region as ECR, e.g. us-east-1
AWS_ROLE_TO_ASSUME	Role ARN from Step 5
ECR_REPO	fgs (optional; workflow default)

Do **not** create:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY

Workflows already request permissions: id-token: write for OIDC.

PUSH_TO_ECR is **on by default** in `.github/workflows/reusable-build-service.yml`. Set the variable to `false` only if you want CI to build without publishing.

Step 7 — Prove the connection

1. Confirm workflows are on the branch you run (dev / test / main):
 - `.github/workflows/build-setup.yml`
 - `.github/workflows/build-user.yml`
 - `.github/workflows/build-nginx.yml`
 - `.github/workflows/reusable-build-service.yml`
2. GitHub → **Actions** → **Build setup** (or user / nginx) → **Run workflow** → branch dev → **force** = true.
3. On success, open **ECR** → **Repositories** → `fgs` → **Images**.

Expected tags (examples):

```
ACCOUNT.dkr.ecr.REGION.amazonaws.com/fgs:setup-dev
ACCOUNT.dkr.ecr.REGION.amazonaws.com/fgs:user-dev
ACCOUNT.dkr.ecr.REGION.amazonaws.com/fgs:nginx-dev
```

Also pushed: `setup-<version>-dev`, `setup-<version>-dev-<sha>` (same pattern for user / nginx).

Pull requests **build only**; they do not push to ECR.

Troubleshooting

Symptom	Likely cause
AssumeRoleWithWebIdentity denied	Trust sub does not match this repo, or OIDC provider missing

Symptom	Likely cause
Login OK, push fails	Inline policy Resource ARN wrong, or repo name is not fgs
Build OK, no image in ECR	Missing AWS_ROLE_TO_ASSUME, or PUSH_TO_ECR=false, or only a PR ran
Empty Repositories list	Wrong region, or repository fgs was never created
Registry host only, no repo	ACCOUNT.dkr.ecr.REGION.amazonaws.com is the registry host — open Repositories and create fgs

Checklist

- ☐ ECR private repository fgs exists in the target region
- ☐ IAM OIDC provider token.actions.githubusercontent.com
- ☐ Role fgs-dev-github-actions with correct trust sub
- ☐ Inline policy allows push to ...:repository/fgs
- ☐ GitHub variables AWS_REGION and AWS_ROLE_TO_ASSUME
- ☐ Workflow run on dev shows images setup-dev / user-dev / nginx-dev